

STUDENT MANAGEMENT SYSTEM

CRUD-Based Full-Stack Web Application

Technology Stack: React + Django REST Framework + SQLite

This report documents the application interface, CRUD operations, validation, search functionality, and backend test execution using the submitted screenshots.

1. Project Overview

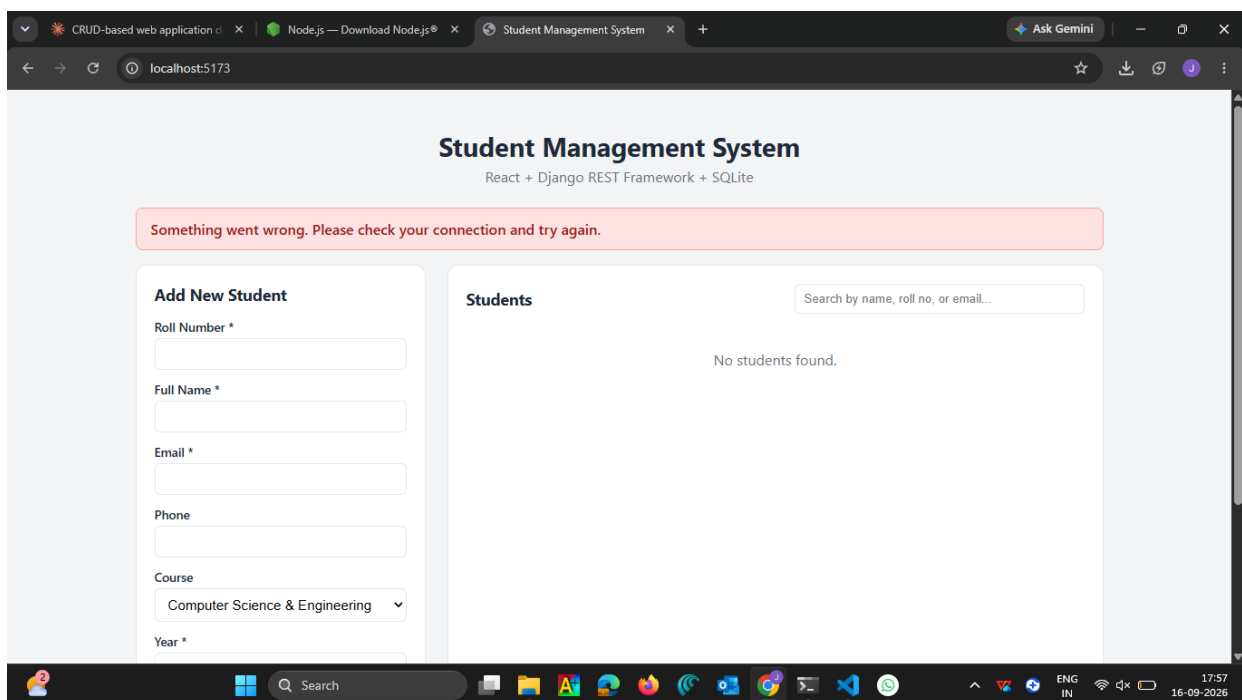
The Student Management System is a full-stack web application designed to manage student records. The interface provides fields for roll number, full name, email, phone, course, year, and GPA. The application supports Create, Read, Update, and Delete (CRUD) operations and provides search and validation features.

2. Technology Stack

Layer	Technology
Frontend	React
Backend	Django REST Framework
Database	SQLite
Testing / Development	Browser + Python/Django test suite

3. Screenshot-Based Demonstration

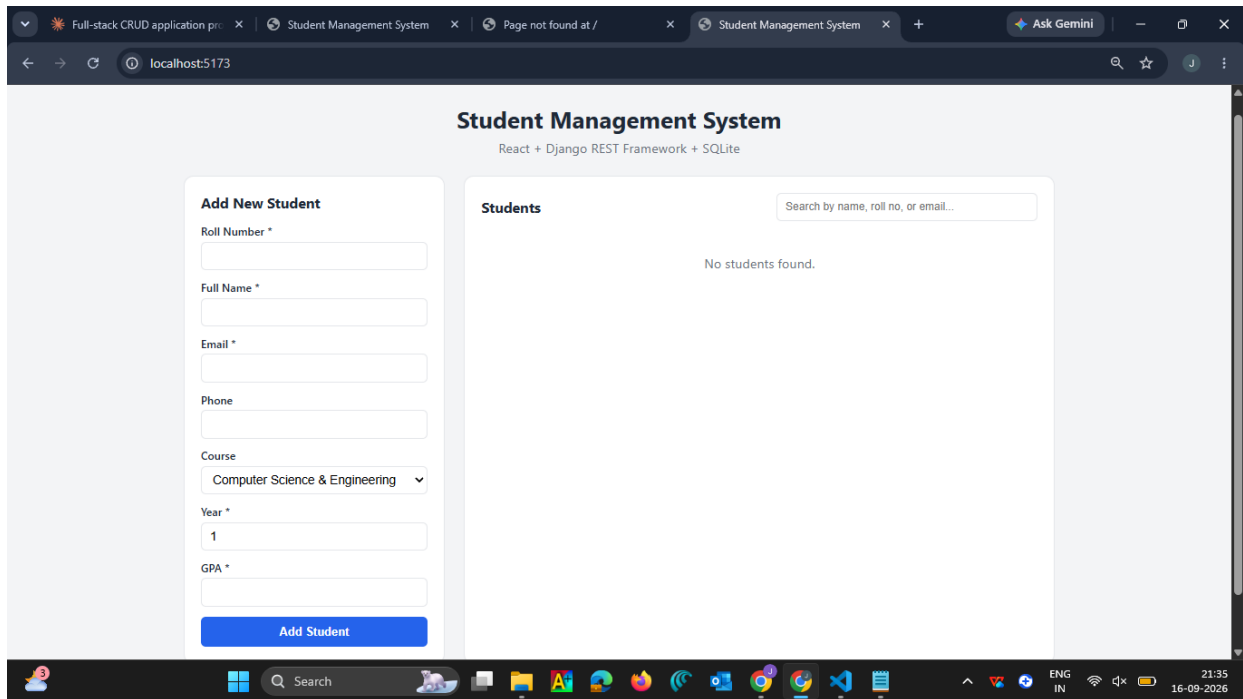
Figure 1 – Initial API/Connection Error



Explanation: The initial application screen displays the Student Management System interface. A red message indicates that the application could not retrieve data at that moment. The student form and student list area are still rendered, showing that the frontend interface is available.

Purpose in SOP: This screenshot documents the initial connection/error-handling state.

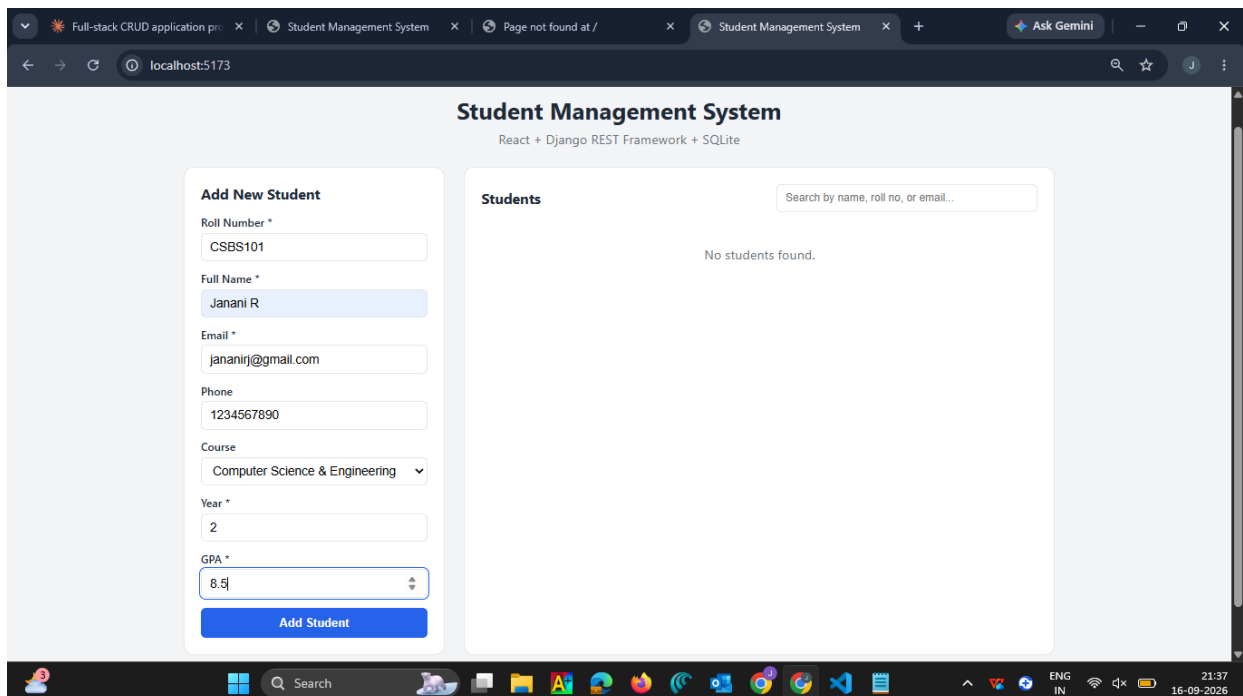
Figure 2 – Empty Student List



Explanation: The application loads successfully and shows the Add New Student form on the left and the Students section on the right. The message 'No students found' indicates that the database currently has no records available to display.

Purpose in SOP: This demonstrates the Read operation when the data set is empty.

Figure 3 – Create Operation



Explanation: A student record has been entered with roll number CSBS101, name Janani R, email, phone, course, year, and GPA. The green message confirms that the student was added successfully, and the new record appears in the Students table.

Purpose in SOP: This demonstrates the Create operation: data is submitted through the form and displayed after successful insertion.

Figure 4 – Duplicate Data Validation

React + Django REST Framework + SQLite

Student added successfully.

Add New Student

Roll Number *
CSBS101

Full Name *
Janani R

Email *
jananirj@gmail.com

Phone
1234567890

Course
Computer Science & Engineering

Year *
2

GPA *
8.5

Add Student

Students

Search by name, roll no, or email...

Roll No	Name	Email	Phone	Course	Year	GPA	Actions
CSBS101	Janani R	jananirj@gmail.com	1234567890	CSE	2	8.50	Edit Delete

Explanation: The application displays a validation error stating that the roll number and email already exist. The existing student remains visible in the table.

Purpose in SOP: This demonstrates server-side validation for unique fields and prevents duplicate student records.

Figure 5 – Edit Student Form

Student Management System

React + Django REST Framework + SQLite

roll_no: student with this roll no already exists. | email: student with this email already exists.

Add New Student

Roll Number *
CSBS101

Full Name *
Janani R

Email *
jananirj@gmail.com

Phone
1234567890

Course
Computer Science & Engineering

Year *
2

GPA *
8.5

Add Student

Students

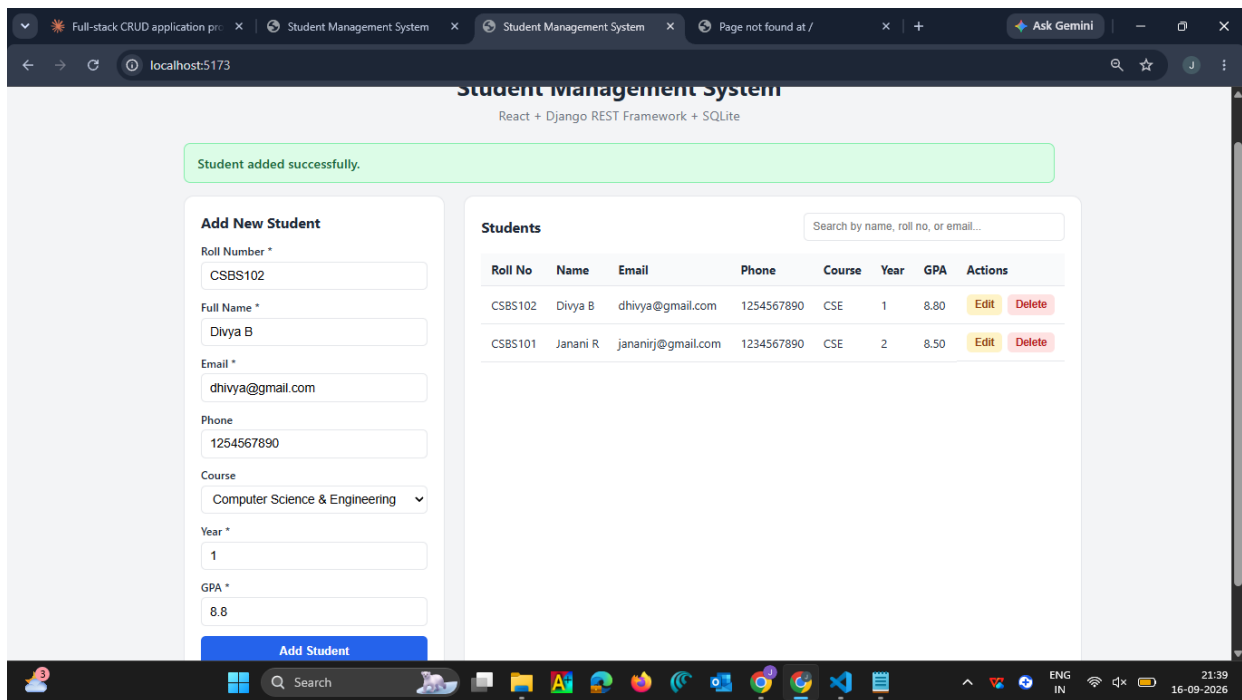
Search by name, roll no, or email...

Roll No	Name	Email	Phone	Course	Year	GPA	Actions
CSBS101	Janani R	jananirj@gmail.com	1234567890	CSE	2	8.50	Edit Delete

Explanation: The Edit Student interface is populated with an existing student's details. The Update Student and Cancel controls are available, allowing the selected record to be modified.

Purpose in SOP: This demonstrates the Update workflow before submitting the changed data.

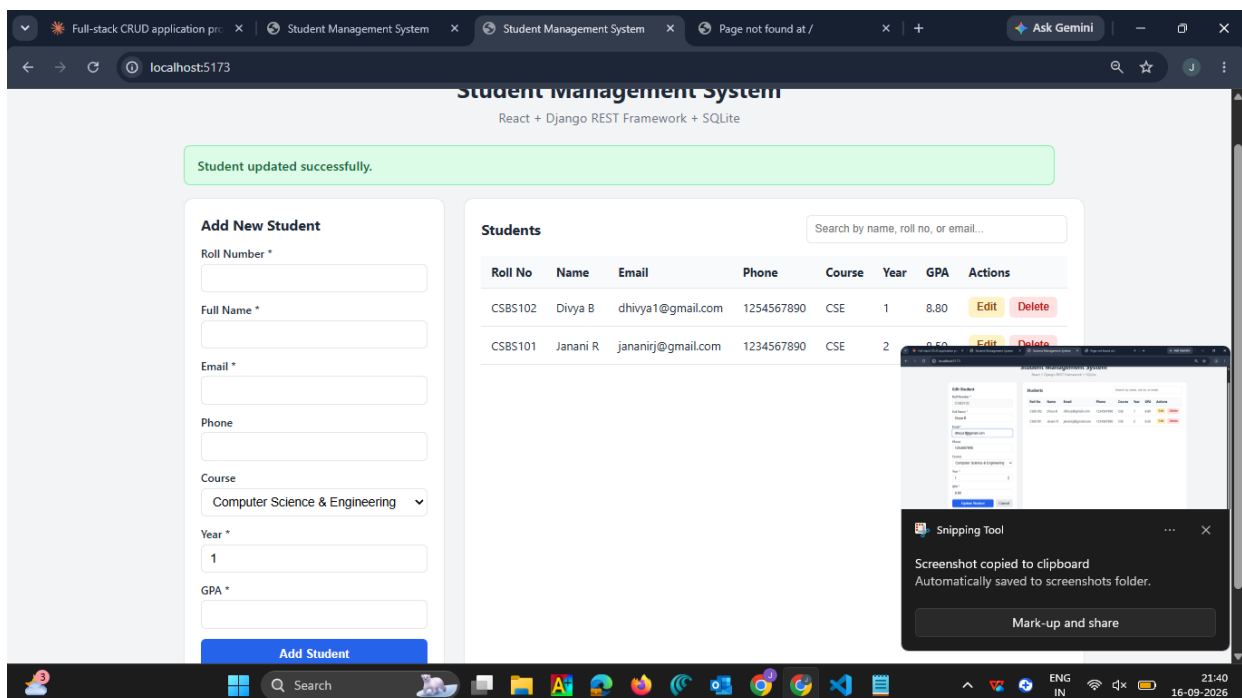
Figure 6 – Successful Update



Explanation: The green success message confirms that the student record was updated successfully. The student table displays the updated information after the operation.

Purpose in SOP: This demonstrates the Update operation and successful refresh of the displayed data.

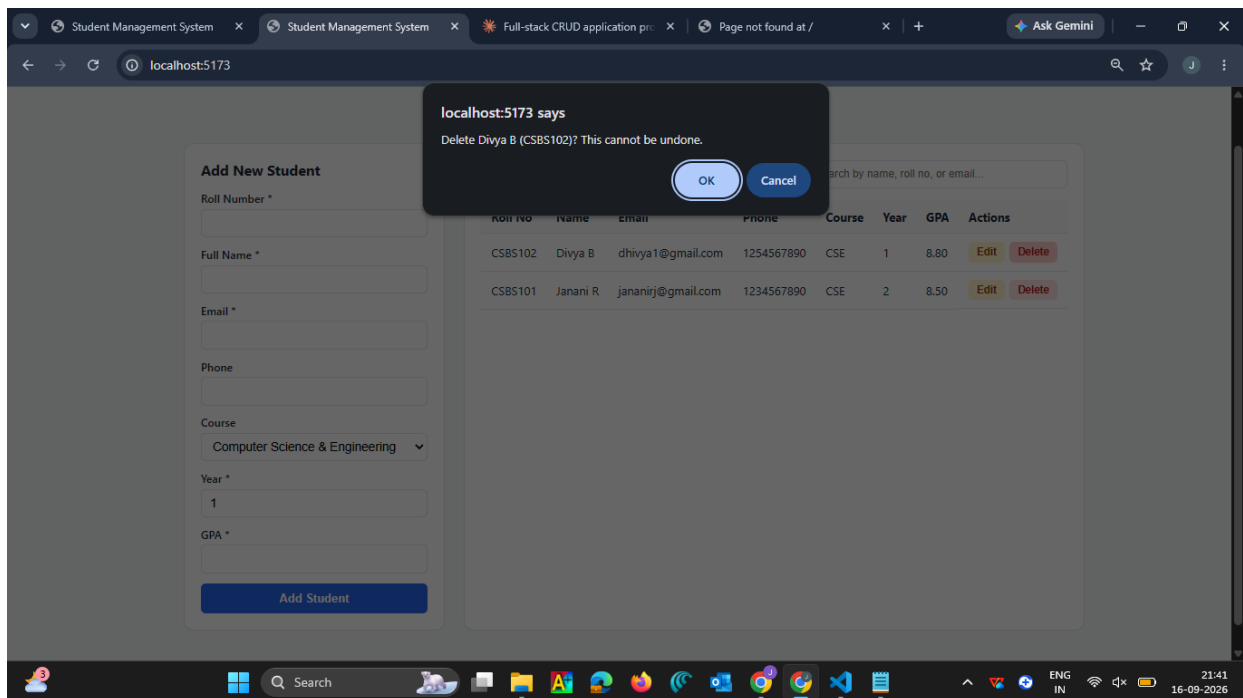
Figure 7 – Successful Delete



Explanation: The application displays a green 'Student deleted successfully' message. The deleted record is no longer present in the student list.

Purpose in SOP: This demonstrates the Delete operation and confirms that the record was removed from the application data set.

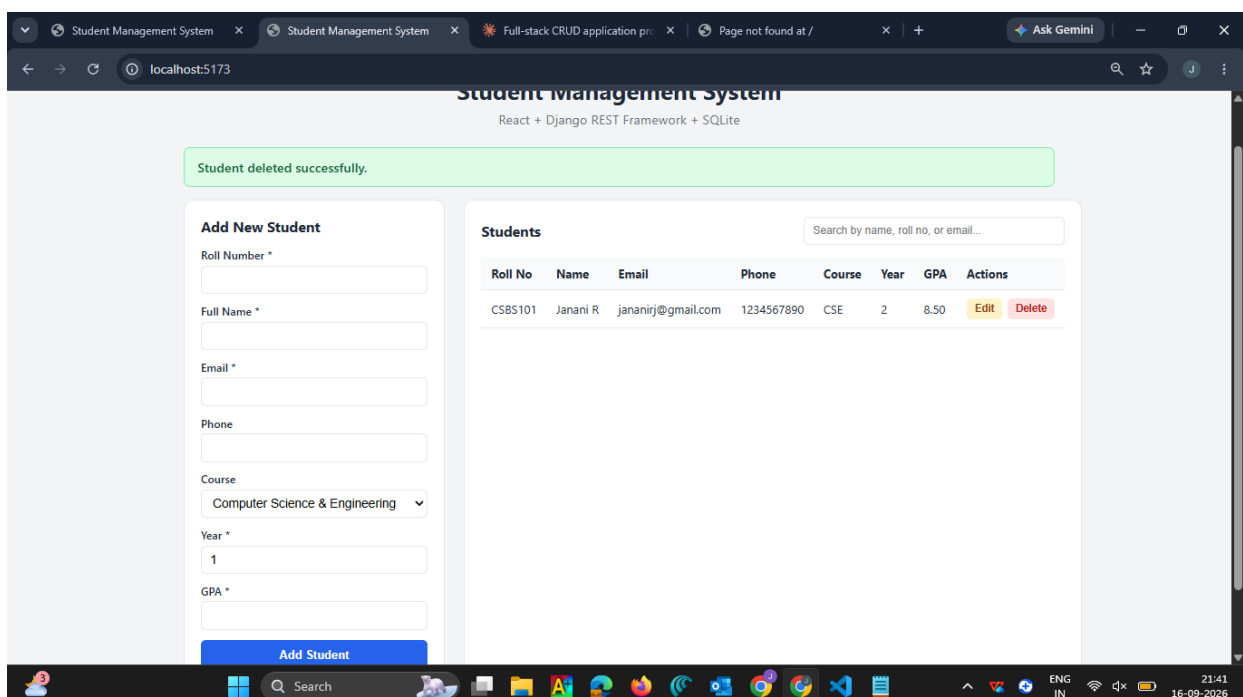
Figure 8 – Search / Filter Functionality



Explanation: A search term is entered in the search box, and the student table is filtered to show the matching record. The result includes the matching student's roll number, name, email, course, year, and GPA.

Purpose in SOP: This demonstrates client-side search/filter functionality for quickly locating student records.

Figure 9 – Student Form Ready for Entry



Explanation: The Add New Student form is displayed with the available fields and course selection. The Students panel is empty in this state, providing a clean form for entering a new record.

Purpose in SOP: This demonstrates the normal input state of the application before a new Create operation.

Figure 10 – Multiple Student Records

The screenshot shows a web browser window with the URL `localhost:5173`. The page title is "Student Management System" with a subtitle "React + Django REST Framework + SQLite".

On the left, there is a form titled "Add New Student" with the following fields:

- Roll Number *:
- Full Name *:
- Email *:
- Phone:
- Course:
- Year *:
- GPA *:
-

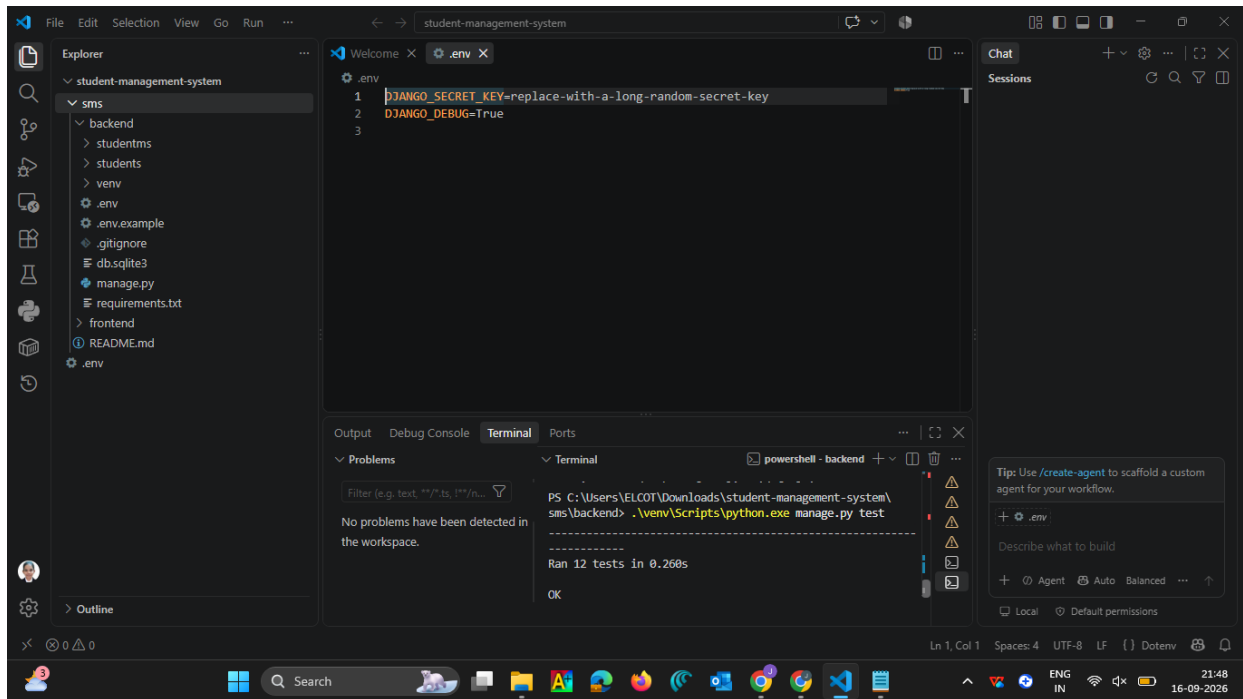
On the right, there is a section titled "Students" with a search bar containing "raju". Below the search bar is a table with the following data:

Roll No	Name	Email	Phone	Course	Year	GPA	Actions
CSBS103	raju	raju@gmail.com	1234567899	CSE	3	8.80	Edit Delete

Explanation: The student table contains multiple records, including CSBS102 and CSBS101. Each row provides Edit and Delete actions, showing that the system can manage more than one record.

Purpose in SOP: This demonstrates the Read operation with a populated database and the availability of row-level CRUD controls.

Figure 11 – Backend Test Execution



Explanation: The development terminal shows the Django test command completing successfully with 'Ran 12 tests' followed by 'OK'. This provides evidence that the automated backend test suite passed.

Purpose in SOP: This demonstrates backend testing and successful execution of the project's automated tests.

4. CRUD Summary

Operation	Evidence from Screenshots	Result
Create	Figure 3 / Figure 10	New student records are added and displayed.
Read	Figure 2 / Figure 10	Student records are listed in the table.
Update	Figure 5 / Figure 6	Existing student details are edited and updated.
Delete	Figure 7	Selected student record is removed.

5. Validation and Testing Summary

- Mandatory form fields are marked with an asterisk and are intended to receive required values.
- Duplicate roll numbers and email addresses are rejected, as shown in Figure 4.
- Delete requires confirmation before the record is removed, as shown during the delete workflow.
- Search functionality allows records to be filtered using student information.
- The backend test suite reports 12 tests executed successfully with an 'OK' result.

6. Conclusion

The screenshots provide evidence that the Student Management System implements the major requirements of a CRUD-based web application. The interface supports student data entry, record listing, editing, deletion, duplicate-data validation, search, and backend testing. The application is therefore suitable for demonstrating the CRUD workflow described in the SOP.